Continuous delivery with Argo CD

Deliver to Kubernetes by letting a controller pull the desired state from a repository: applications, sync and drift, Helm and Kustomize, environments, hooks, access and debugging.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Pull-based delivery and the repository as source of truth (12 min)
2. Installing Argo CD and the Application resource (13 min)
3. Sync, health and drift (13 min)
4. Helm and Kustomize with Argo CD (12 min)
5. Environments, app-of-apps and ApplicationSets (13 min)
6. Hooks, sync waves and progressive delivery (13 min)
7. Access, projects, secrets and multi-cluster (14 min)
8. Debugging Argo CD (14 min)
9. Argo CD: SSO, RBAC and audit (12 min)
10. Argo CD: notifications and image updates (12 min)
11. Progressive delivery with Argo Rollouts (13 min)
12. Argo CD: disaster recovery and upgrades (12 min)
Cheat sheet

Lesson 1. Pull-based delivery and the repository as source of truth

12 minutes. Understand why many teams let a controller in the cluster pull the desired state from a repository, instead of a pipeline pushing changes in.

In push-based delivery a pipeline holds cluster credentials and runs kubectl or helm against the cluster. In pull-based delivery a controller inside the cluster watches a repository and makes the cluster match what is stored there. The second approach changes who holds credentials, how drift is handled and how you audit changes. Argo CD is the most widely used implementation, and this lesson explains the idea before the tool.

What you will learn

- Contrast push and pull delivery
- Explain the repository as the source of truth
- Describe reconciliation and drift
- Know when this model fits and when it does not

Push and pull

Push: a CI job builds an image, then runs `kubectl apply` or `helm upgrade` using credentials for the cluster. The job is the only place that knows the result, and credentials must be stored in the CI system. **Pull:** CI builds the image and commits a change to a configuration repository (for example, a new image tag). A controller in the cluster notices the commit and applies it. The cluster reaches out; nothing needs inbound access or stored cluster credentials in CI.

The repository as source of truth

The desired state of the cluster is written as manifests, Helm charts or Kustomize overlays in a repository. A change is a commit, reviewed in a pull request, with an author, a message and a history. Rolling back is reverting a commit. The history answers "who changed this, when and why?" without searching through pipeline logs.

Reconciliation and drift

The controller continuously compares **desired state** (the repository) with **live state** (the cluster). If someone edits a resource by hand, the two diverge: that is **drift**. The controller can report it, or put things back. This turns manual hot-fixes from invisible changes into visible differences.

Trade-offs

Benefits: audit trail, consistent environments, simple rollback, fewer credentials in CI, and recovery by pointing a new cluster at the repository. Costs: a separate configuration repository to manage, a delay between commit and rollout, secrets need special handling because the repository must not contain them in plain text, and more concepts for newcomers. It fits Kubernetes well; it fits less well for one-off scripts or resources outside the cluster.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Pull-based delivery and the repository as source... fits together

Part	What it does	What goes wrong
Push and pull	Push: a CI job builds an image, then runs kubectl apply or helm upgrade using credentials for the cluster.	Mixing application code and cluster configuration without a plan.
The repository as source of truth	The desired state of the cluster is written as manifests, Helm charts or Kustomize overlays in a repository.	Treating the repository as optional.
Reconciliation and drift	The controller continuously compares desired state (the repository) with live state (the cluster).	Hand-editing the cluster and expecting it to stay.
Trade-offs	Benefits: audit trail, consistent environments, simple rollback, fewer credentials in CI, and recovery by pointing a new cluster at the repository.	Putting plain-text secrets in it.

Flow: Push and pull to The repository as source of truth / The repository as source of truth to Reconciliation and drift / Reconciliation and drift to Trade-offs

Incident: The hot-fix that vanished

> **Incident:** *An engineer scaled a deployment up by hand during a spike, and twenty minutes later it returned to 2 replicas.*

```
$ kubectl scale deploy/api --replicas=10 -n shop
$ kubectl get deploy api -n shop
NAME  READY  UP-TO-DATE
api   2/2    2
```

Automatic self-healing restored the repository value. The team learned to change replicas by commit or by autoscaler, and agreed on an emergency procedure: pause automation for that application, fix, then commit the same change.

Common mistakes

- Treating the repository as optional
- Putting plain-text secrets in it
- Hand-editing the cluster and expecting it to stay
- Mixing application code and cluster configuration without a plan

Check yourself

What does a pull-based controller compare?

Answer: Desired state in a repository with live state in the cluster

What is drift?

Answer: Live state differing from desired state

How do you roll back?

Answer: Revert the commit

Why does this reduce credentials in CI?

Answer: CI does not need cluster access; the controller pulls

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Installing Argo CD and the Application resource

13 minutes. Install Argo CD in a cluster, sign in, and describe an application with the Application custom resource.

Argo CD runs in your cluster as a set of components: an API server and UI, a repository server that renders manifests, and an application controller that compares and syncs. You tell it what to deliver with a custom resource called an Application, which names a source in a repository and a destination in a cluster.

What you will learn

- Install Argo CD and reach its UI
- Name the main components
- Write an Application manifest
- Understand source, destination and project

Install

Create a namespace and apply the official install manifest (or use the Helm chart):

```
kubectl create namespace argocd
kubectl apply -n argocd -f install.yaml   # the stable install manifest from the Getting started p \
    age
kubectl -n argocd get pods
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 - \
    d
kubectl -n argocd port-forward svc/argocd-server 8080:443
```

The initial admin password is in a secret; change it or switch to single sign-on and delete the secret. For production, pin a version, run it highly available and put the UI behind TLS and an ingress.

Components

The **API server** serves the UI, CLI and API and handles authentication. The **repo server** clones repositories and renders manifests from plain YAML, Helm or Kustomize. The **application controller** compares the rendered desired state with live state and performs syncs. Redis caches state and optional components handle notifications and ApplicationSets.

The Application resource

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://example.com/platform/shop-config.git
    targetRevision: main
    path: apps/shop-api
  destination:
    server: https://kubernetes.default.svc
    namespace: shop
  syncPolicy:
    syncOptions:
      - CreateNamespace=true
```

The **source** says which repository, revision and folder. The **destination** says which cluster and namespace. The **project** groups applications and limits what they may do. With no automated sync policy, you press Sync (or run `argocd app sync`) yourself.

MARGA by EknathaLabs marga.eknathalabs.com

CLI and UI

The CLI logs in with `argocd login`, lists applications with `argocd app list` and shows differences with `argocd app diff shop-api`. The UI displays the resource tree, sync and health status, and events. Both use the same API and the same permissions.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Installing Argo CD and the Application resource fits together

Part	What it does	What goes wrong
Install	Create a namespace and apply the official install manifest (or use the Helm chart).	Leaving the initial admin password in place.
Components	The API server serves the UI, CLI and API and handles authentication.	Running the default project with no limits.
The Application resource	The source says which repository, revision and folder.	Using a write-capable credential to read a repository.
CLI and UI	The CLI logs in with argocd login, lists applications with argocd app list and shows differences with argocd app diff shop-api.	Installing without pinning a version.

Flow: Install to Components / Components to The Application resource / The Application resource to CLI and UI

Incident: Unknown repository

Incident: *A new application showed "ComparisonError: repository not accessible".*

```
$ argocd app get shop-api | head -8
Sync Status: Unknown
Condition: ComparisonError: rpc error: authentication required
$ argocd repo list
TYPE  NAME  REPO  STATUS
(none)
```

The repository was private and had not been registered. The team added it with a read-only deploy credential through the repository settings, and the application compared successfully. They kept credentials in the cluster as secrets and not in the Application manifest.

Common mistakes

- Leaving the initial admin password in place
- Running the default project with no limits
- Using a write-capable credential to read a repository
- Installing without pinning a version

Check yourself

What does the repo server do?

Answer: Fetches and renders manifests

What does an Application name?

Answer: A source and a destination

What does the controller do?

Answer: Compares desired with live state and syncs

What credential does a repository need?

Answer: Read-only access

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. Sync, health and drift

13 minutes. Read sync and health status, choose manual or automated sync, and understand prune and self-heal.

Argo CD shows two separate statuses for every application. Sync status says whether live state matches the repository. Health status says whether the resources actually work. They are independent, and reading them together tells you what to do next. Automation options such as prune and self-heal are powerful and need care.

What you will learn

- Read Synced, OutOfSync, Healthy, Progressing and Degraded
- Choose a sync policy
- Understand prune and self-heal
- Use sync options and ignore rules for expected differences

Two statuses

Sync status: Synced (live matches desired), OutOfSync (they differ) or Unknown (cannot compare). **Health:** Healthy, Progressing (rolling out), Degraded (failing), Suspended, Missing. An application can be Synced but Degraded: the right manifests are applied but the pods crash. It can be OutOfSync but Healthy: someone changed a replica count and the app still works.

Manual or automated

With manual sync, Argo CD shows differences and waits for a person or pipeline to sync. With **automated** sync, it applies new commits on its own. Two further switches matter. **Prune** deletes resources that are no longer in the repository. **Self-heal** reverts live changes made outside the repository.

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

Use automation for non-production first. For production, many teams automate but keep prune off until they trust their repository layout, or require a manual sync for risky applications.

Why prune needs care

Prune removes anything Argo CD manages that disappears from the source. A renamed folder, a wrong path or an empty render can look like "delete everything". Protect critical resources with the annotation `argocd.argoproj.io/sync-options: Prune=false`, and use sync windows or manual approval for sensitive applications.

Expected differences

Controllers add fields, such as an autoscaler changing replicas or a webhook injecting certificates. Tell Argo CD to ignore them with `ignoreDifferences` for those paths, or the application will show OutOfSync forever. Sync options like `ServerSideApply=true` help when several controllers manage one object.

Retrying

A sync can fail on a bad manifest. Use `retry` settings for transient problems, and read the sync result message for permanent ones. Argo CD records sync history so you can see what was applied at each revision.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Sync, health and drift fits together

Part	What it does	What goes wrong
Two statuses	Sync status: Synced (live matches desired), OutOfSync (they differ) or Unknown (cannot compare).	Reading only sync status and ignoring health.
Manual or automated	With manual sync, Argo CD shows differences and waits for a person or pipeline to sync.	Turning on prune everywhere on day one.
Why prune needs care	Prune removes anything Argo CD manages that disappears from the source.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Expected differences	Controllers add fields, such as an autoscaler changing replicas or a webhook injecting certificates.	Fighting controllers instead of ignoring fields.
Retrying	A sync can fail on a bad manifest. Use retry settings for transient problems, and read the sync result message for permanent ones.	No review for repository restructures.

Flow: Two statuses to Manual or automated / Manual or automated to Why prune needs care / Why prune needs care to Expected differences / Expected differences to Retrying

Incident: The prune that removed a namespace

Incident: *After a folder rename, an automated application with prune turned on deleted its resources.*

```
$ argocd app get shop-api --refresh | head -6
Sync Status: OutOfSync
Sync Policy: Automated (Prune)
$ argocd app diff shop-api | head -3
===== apps/Deployment shop/api ======
< (deleted in source)
```

The path in the Application no longer matched the folder, so the render was empty and prune removed everything. The team restored the path, resynced, then set Prune=false on stateful resources and added a pre-merge check that renders the manifests.

Common mistakes

- Turning on prune everywhere on day one
- Reading only sync status and ignoring health
- Fighting controllers instead of ignoring fields
- No review for repository restructures

Check yourself

What does self-heal do?

Answer: Reverts live changes made outside the repository

What does prune do?

Answer: Deletes resources removed from the source

What can OutOfSync plus Healthy mean?

Answer: Live state was changed but the app works

How do you ignore fields changed by other controllers?

Answer: ignoreDifferences

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Helm and Kustomize with Argo CD

12 minutes. Deliver Helm charts and Kustomize overlays, set values per environment, and understand that Argo CD renders rather than installs.

Most real repositories use Helm charts, Kustomize overlays or both. Argo CD renders them itself and applies the result as plain manifests. This has consequences: Helm hooks and release records work differently, and values must live in the repository to stay reviewable.

What you will learn

- Point an Application at a Helm chart or a Kustomize folder
- Set values per environment
- Know how Argo CD treats Helm releases
- Pin chart versions

Helm sources

The Application source can be a chart in a Helm repository or a folder in the repository. Values come from a values file or inline parameters.

```
source:
  repoURL: https://charts.example.com
  chart: shop-api
  targetRevision: 1.4.2
  helm:
    valueFiles:
      - values-prod.yaml
    parameters:
      - name: replicaCount
        value: "4"
```

Argo CD runs the equivalent of `helm template` and applies the output. It does not run `helm install`, so `helm list` will not show the release and Helm release history is not used. Argo CD tracks history itself.

Kustomize

Point at a folder with a `kustomization.yaml`. An overlay per environment patches a shared base: change replicas, resource limits, image tags and config. `kustomize edit set image` or the Application's `kustomize.images` field updates the image.

Keep values in the repository

Inline overrides set in the UI become live state that nobody reviewed. Prefer values files committed to the repository, one per environment, so a diff in a pull request shows exactly what changes in production.

Pin versions

Use an exact chart version, not a floating range or `HEAD`. Update by a pull request that shows the rendered difference. Tools that render manifests in the pull request make review easy.

Hooks and tests

Helm hooks are converted into Argo CD hooks where possible, but behaviour differs from Helm's release lifecycle. Test the render locally with `helm template` or `kustomize build` and compare with `argocd app diff`.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Helm and Kustomize with Argo CD fits together

Part	What it does	What goes wrong
Helm sources	The Application source can be a chart in a Helm repository or a folder in the repository.	Assuming Helm release commands work.
Kustomize	Point at a folder with a kustomization.yaml.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Keep values in the repository	Inline overrides set in the UI become live state that nobody reviewed.	Overrides outside the repository.
Pin versions	Use an exact chart version, not a floating range or HEAD.	Floating chart versions.
Hooks and tests	Helm hooks are converted into Argo CD hooks where possible, but behaviour differs from Helm's release lifecycle.	Not rendering locally before merge.

Flow: Helm sources to Kustomize / Kustomize to Keep values in the repository / Keep values in the repository to Pin versions / Pin versions to Hooks and tests

Incident: The value nobody reviewed

Incident: *A parameter was changed in the UI to hotfix replicas, and a later commit silently reverted it.*

```
$ argocd app get shop-api -o yaml | grep -A3 parameters
    parameters:
    - name: replicaCount
      value: "8"
$ grep replicaCount apps/shop-api/values-prod.yaml
replicaCount: 4
```

UI parameters override files, so the hotfix existed only in the application object. The team moved the value into the file, removed the override, and made the UI read-only for parameters on production.

Common mistakes

- Overrides outside the repository
- Floating chart versions
- Assuming Helm release commands work
- Not rendering locally before merge

Check yourself

Does Argo CD run helm install?

Answer: No, it renders and applies manifests

Where should values live?

Answer: In files in the repository

What should the chart version be?

Answer: Pinned exactly

What do Kustomize overlays do?

Answer: Patch a base per environment

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Environments, app-of-apps and ApplicationSets

13 minutes. Organise many applications and environments with a repository layout, the app-of-apps pattern and ApplicationSets.

One application is easy. Fifty applications across dev, staging and production, and several clusters, need structure. The goal is that adding an application or an environment is a small, reviewable change, and promotion between environments is a commit.

What you will learn

- Choose a repository layout
- Use the app-of-apps pattern
- Generate applications with ApplicationSets
- Promote changes between environments

Layout

A common layout has a folder per application with a base and overlays per environment, or a folder per environment that points at shared charts. Keep application source code and configuration in separate repositories when possible, so a new image tag is a small commit that does not trigger application tests.

App-of-apps

A parent Application points at a folder full of Application manifests. Syncing the parent creates all the children. This lets you bootstrap a whole cluster from one command, and add a new application by adding one file.

ApplicationSets

An **ApplicationSet** generates Applications from a template and a generator. A list generator makes one per environment; a cluster generator makes one per registered cluster; a directory generator makes one per folder in the repository; a pull request generator can create temporary preview environments.

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: shop
  namespace: argocd
spec:
  generators:
    - list:
        elements:
          - env: staging
          - env: production
  template:
    metadata:
      name: "shop-{{env}}"
    spec:
      project: default
      source:
        repoURL: https://example.com/platform/shop-config.git
        targetRevision: main
        path: "overlays/{{env}}"
      destination:
        server: https://kubernetes.default.svc
        namespace: "shop-{{env}}"
```

Promotion

Promote by copying a tested image tag or chart version from the staging overlay to production in a pull request, ideally automated after checks pass. Avoid promoting by branch merges that carry unrelated changes. Production should require review and a clear record.

MARGA by EknathaLabs marga.eknathalabs.com

Blast radius

A generator that is mistaken can create or remove many applications at once. Use projects, review for ApplicationSet changes, and policies that limit deletion (for example, preserving resources when an application is removed).

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Environments, app-of-apps and ApplicationSets fits together

Part	What it does	What goes wrong
Layout	A common layout has a folder per application with a base and overlays per environment, or a folder per environment that points at shared charts.	One giant repository folder with no structure.
App-of-apps	A parent Application points at a folder full of Application manifests.	Copy-pasted Application files.
ApplicationSets	An ApplicationSet generates Applications from a template and a generator.	No review on generators.
Promotion	Promote by copying a tested image tag or chart version from the staging overlay to production in a pull request, ideally automated after checks pass.	Promoting with merges that carry other changes.
Blast radius	A generator that is mistaken can create or remove many applications at once.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Layout to App-of-apps / App-of-apps to ApplicationSets / ApplicationSets to Promotion / Promotion to Blast radius

Incident: Fifty applications gone

> **Incident:** *A change to an ApplicationSet filter matched nothing and the controller started removing the generated applications.*

```
$ argocd appset get shop | head -4
Name: shop
Generators: git directories: apps/*
$ argocd app list | wc -l
2
```

A folder rename left the generator with no matches. The team restored the path, set the policy that keeps generated applications when the set changes, and required a second reviewer on ApplicationSet files.

Common mistakes

- One giant repository folder with no structure
- Copy-pasted Application files
- Promoting with merges that carry other changes
- No review on generators

Check yourself

What is app-of-apps?

Answer: A parent Application that creates child Applications

What does an ApplicationSet do?

Answer: Generates Applications from a template

How should production promotion happen?

Answer: By a reviewed commit

MARGA by EknathaLabs
marga.eknathalabs.com

Why separate application and configuration repositories?

Answer: Config changes should not trigger code pipelines

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Hooks, sync waves and progressive delivery

13 minutes. Order resources in a sync, run jobs before and after it, and roll out gradually with canary and blue-green strategies.

Real releases have order: create the namespace and the database schema before the application, run a migration job before pods start, smoke-test afterwards. Argo CD provides hooks and sync waves for this, and a companion project, Argo Rollouts, adds canary and blue-green rollouts with automated analysis.

What you will learn

- Order resources with sync waves
- Run jobs with hooks
- Describe canary and blue-green strategies
- Explain how analysis can stop a bad rollout

Sync waves

Annotate resources with `argocd.argoproj.io/sync-wave: "N"`. Lower numbers sync first, and Argo CD waits for each wave to be healthy before starting the next. Default is wave 0. Use negative waves for namespaces and CRDs, and higher ones for things that depend on the application.

Hooks

A hook is a resource, usually a Job, annotated with `argocd.argoproj.io/hook`. **PreSync** runs before the main sync (database migration), **Sync** during it, **PostSync** afterwards (smoke test), **SyncFail** when it fails. Set `hook-delete-policy` such as `HookSucceeded` so completed jobs do not pile up. Make hooks idempotent, because syncs can be retried.

Canary and blue-green

A plain Deployment does a rolling update with no view of user impact. **Argo Rollouts** replaces it with a Rollout resource. A **canary** sends a small share of traffic (5 percent, then 25, 50, 100) to the new version with pauses. **Blue-green** runs the new version beside the old and switches traffic at once after checks, keeping the old for fast rollback. Traffic splitting uses an ingress controller or a service mesh.

Automated analysis

An AnalysisTemplate queries metrics, for example the error rate in Prometheus. If the new version breaks the threshold during a canary step, the rollout aborts and traffic returns to the stable version without a person. This ties delivery to your service level objectives.

Keep it simple first

Start with sync waves and a PreSync migration. Add canary only when you have good metrics and enough traffic to see a signal in minutes.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Hooks, sync waves and progressive delivery fits together

Part	What it does	What goes wrong
Sync waves	Annotate resources with argocd.argoproj.io/sync-wave: "N".	Waves that deadlock on health.
Hooks	A hook is a resource, usually a Job, annotated with argocd.argoproj.io/hook.	Non-idempotent hooks.
Canary and blue-green	A plain Deployment does a rolling update with no view of user impact.	Canary without metrics.
Automated analysis	An AnalysisTemplate queries metrics, for example the error rate in Prometheus.	If the new version breaks the threshold during a canary step, the rollout aborts and traffic returns to the stable version without a person.
Keep it simple first	Start with sync waves and a PreSync migration.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Sync waves to Hooks / Hooks to Canary and blue-green / Canary and blue-green to Automated analysis / Automated analysis to Keep it simple first

Incident: The migration that ran twice

Incident: *A retried sync ran a PreSync migration a second time and failed on a duplicate column.*

```
$ kubectl logs job/migrate -n shop
ERROR: column "currency" of relation "orders" already exists
$ kubectl get job migrate -n shop -o yaml | grep hook
argocd.argoproj.io/hook: PreSync
```

The migration was not idempotent. The team rewrote it to check for the column first, recorded versions in a migrations table, and set the hook delete policy so each run starts clean.

Common mistakes

- Non-idempotent hooks
- Waves that deadlock on health
- Canary without metrics
- No rollback plan for data changes

Check yourself

What do sync waves control?

Answer: The order resources are applied

When does a PreSync hook run?

Answer: Before the main sync

What is a canary?

Answer: Sending a small share of traffic to the new version first

What can automated analysis do?

Answer: Abort a rollout when metrics break a threshold

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Access, projects, secrets and multi-cluster

14 minutes. Limit who can do what with projects and RBAC, keep secrets out of the repository, and deliver to more than one cluster.

Argo CD is powerful: it can create anything in the clusters it manages, so protecting it matters. Three topics decide whether a shared installation is safe: who may do what (projects, RBAC and single sign-on), where secrets live, and how remote clusters are registered.

What you will learn

- Use projects to limit sources, destinations and kinds
- Configure RBAC and single sign-on
- Handle secrets without storing them in plain text
- Register and secure multiple clusters

Projects

An **AppProject** limits which repositories an application may use, which clusters and namespaces it may deploy to, and which resource kinds it may create (for example, no cluster roles). Give each team a project. Do not run everything in `default`, which is unrestricted.

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: team-shop
  namespace: argocd
spec:
  sourceRepos: ["https://example.com/platform/shop-config.git"]
  destinations:
    - server: https://kubernetes.default.svc
      namespace: "shop-*"
  clusterResourceWhitelist: []
```

RBAC and sign-in

Connect single sign-on (OpenID Connect) so people use company identities and MFA, and disable the shared admin account. Map groups to roles in `argocd-rbac-cm`: viewers see, developers sync their project, platform administrators manage projects. Use project roles and tokens for automation with limited scope.

Secrets

The repository must not hold plain secrets. Common approaches: **sealed or encrypted secrets** committed in encrypted form, **External Secrets Operator** that creates Kubernetes Secrets from a cloud secrets manager, or a **plugin** that injects values at render time. Choose one and standardise it. Remember that Argo CD itself stores cluster credentials and repository credentials as secrets, so protect its namespace.

Multiple clusters

Register a remote cluster with `argocd cluster add`, which creates a service account in that cluster and stores its token in Argo CD. Prefer a hub model with one control cluster and narrow roles per target, or one Argo CD per environment tier so a compromise of non-production cannot reach production. Limit what the remote service account can do where possible.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Access, projects, secrets and multi-cluster fits together

Part	What it does	What goes wrong
Projects	An AppProject limits which repositories an application may use, which clusters and namespaces it may deploy to, and which resource kinds it may create (for example, no...	Using the default project for everything.
RBAC and sign-in	Connect single sign-on (OpenID Connect) so people use company identities and MFA, and disable the shared admin account.	Shared admin login.
Secrets	The repository must not hold plain secrets.	Plain secrets in values files.
Multiple clusters	Register a remote cluster with argocd cluster add, which creates a service account in that cluster and stores its token in Argo CD.	One Argo CD with unlimited reach into production.

Flow: Projects to RBAC and sign-in / RBAC and sign-in to Secrets / Secrets to Multiple clusters

Incident: Secret in the repository

Incident: *A scan found a database password in a values file inside the configuration repository.*

```
$ grep -rn "password:" apps/shop-api/values-prod.yaml
12:  password: "Pr0d-Db-2024!"
$ git log --oneline -- apps/shop-api/values-prod.yaml | wc -l
9
```

The password existed in nine commits. The team rotated it first, since history cannot be trusted to hide it, then replaced the value with a reference to the External Secrets Operator, added secret scanning in pull requests, and restricted access to the repository.

Common mistakes

- Using the default project for everything
- Shared admin login
- Plain secrets in values files
- One Argo CD with unlimited reach into production

Check yourself

What does an AppProject limit?

Answer: Sources, destinations and resource kinds

Where should secrets be stored?

Answer: Outside the repository or encrypted

Why use single sign-on?

Answer: Central identities, MFA and offboarding

What do you do first if a secret was committed?

Answer: Rotate it

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Debugging Argo CD

14 minutes. Find out why an application will not sync, stays OutOfSync, or turns Degraded, and what to check in order.

Argo CD problems look like a handful of patterns: it cannot read the repository, it cannot render the manifests, it cannot apply them, or the applied resources are unhealthy. Walking those four stages in order finds almost every cause.

What you will learn

- Diagnose repository and credential errors
- Diagnose render failures
- Diagnose apply and permission failures
- Diagnose health problems and endless OutOfSync

Four stages

1 **Fetch:** can the repo server clone the repository and find the path and revision? 2. **Render:** does Helm or Kustomize produce valid manifests with the given values? 3. **Apply:** are the manifests accepted by the API server, and does the destination allow them? 4. **Health:** do the created resources become healthy? `argocd app get NAME` shows conditions for the first failing stage.

Fetch and render

Errors like "authentication required", "revision not found" or "app path does not exist" are fetch problems: check repository credentials, branch names and the folder. Render errors show the Helm or Kustomize message ("nil pointer evaluating", "unable to find one 'resource'"): reproduce locally with `helm template` or `kustomize build` using the same values. Check that the repo server has network access to chart repositories.

Apply failures

"is forbidden" means the Argo CD service account lacks permission; "not permitted in project" means the project does not allow that kind or destination; webhook or admission errors come from the cluster; "immutable field" needs the resource to be replaced; "CRD not found" means ordering: sync the CRD first using a wave. Use `argocd app sync NAME --dry-run` and `argocd app diff` to see what would be applied.

Health and OutOfSync loops

A Degraded application needs ordinary Kubernetes debugging on the resources: events, logs, probes. An application that is OutOfSync again right after a sync usually has another controller changing a field (add `ignoreDifferences`), a mutating webhook, or defaults added by the API server. Compare the live and desired YAML for the differing field.

Useful commands

`argocd app get NAME`, `argocd app diff NAME`, `argocd app history NAME`, `argocd app logs NAME`, `kubectl -n argocd logs deploy/argocd-repo-server`, and `kubectl -n argocd logs statefulset/argocd-application-controller`. To roll back, revert the commit; the `argocd app rollback` command is limited when automatic sync is on.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Debugging Argo CD fits together

Part	What it does	What goes wrong
Four stages	Fetch: can the repo server clone the repository and find the path and revision?	Not reproducing the render locally.
Fetch and render	Errors like "authentication required", "revision not found" or "app path does not exist" are fetch problems: check repository credentials, branch names and the folder.	Forcing syncs with replace without understanding the cause.
Apply failures	"is forbidden" means the Argo CD service account lacks permission; "not permitted in project" means the project does not allow that kind or destination; webhook or...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Health and OutOfSync loops	A Degraded application needs ordinary Kubernetes debugging on the resources: events, logs, probes.	Changing things before reading the condition.
Useful commands	argocd app get NAME, argocd app diff NAME, argocd app history NAME, argocd app logs NAME, kubectl -n argocd logs deploy/argocd-repo-server, and kubectl -n argocd logs...	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Four stages to Fetch and render / Fetch and render to Apply failures / Apply failures to Health and OutOfSync loops / Health and OutOfSync loops to Useful commands

Incident: Forever OutOfSync

Incident: *An application showed OutOfSync after every sync although nothing had changed in the repository.*

```
$ argocd app diff shop-api
===== apps/Deployment shop/api ======
  spec:
    replicas:
<     2
>     5
```

A horizontal autoscaler was setting replicas to 5 while the repository said 2. The team removed replicas from the manifest and added ignoreDifferences for that field, so each tool owned its own setting and the application stayed Synced.

Common mistakes

- Changing things before reading the condition
- Not reproducing the render locally
- Fighting autoscalers
- Forcing syncs with replace without understanding the cause

Check yourself

Which command shows conditions for an application?

Answer: argocd app get

What does "app path does not exist" mean?

Answer: The folder or revision in the source is wrong

Why might an app always show OutOfSync?

Answer: Another controller changes a field

How do you roll back with automation on?

Answer: Revert the commit

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 9. Argo CD: SSO, RBAC and audit

12 minutes. Connect Argo CD to your identity provider, define roles and project permissions and keep an audit trail of who changed what.

Argo CD starts with a shared admin account, which is fine for a first test and wrong for a team. Production use needs identity, roles and evidence.

What you will learn

- OIDC single sign-on and removing local admin
- Writing RBAC policy for roles and groups
- Limiting scope with AppProjects
- Audit trails and monitoring

SSO

```
# argocd-cm
data:
  url: https://argocd.example.com
  oidc.config: |
    name: Company SSO
    issuer: https://login.example.com
    clientID: argocd
    clientSecret: $oidc.clientSecret
    requestedScopes: ["openid", "profile", "email", "groups"]
  admin.enabled: "false"
```

Store the client secret in the Argo CD secret, not in the config map. After SSO works, disable the local `admin` account.

RBAC

```
# argocd-rbac-cm: policy.csv
p, role:dev, applications, get, shop/*, allow
p, role:dev, applications, sync, shop/*, allow
p, role:release, applications, override, shop/*, allow
g, shop-developers, role:dev
g, platform-oncall, role:admin
```

Policies read as subject, resource, action, object. The object is usually `project/application`. Map identity provider groups to roles, and default to read-only for everyone else with `policy.default: role:readonly`.

Projects

An **AppProject** limits which Git repositories an application may use, which clusters and namespaces it may deploy to and which Kubernetes kinds it may create. This stops a team from deploying to another team's namespace or creating cluster-wide resources.

```
spec:
  sourceRepos: ["https://git.example.com/shop/*"]
  destinations:
    - {server: https://kubernetes.default.svc, namespace: shop-*}
  clusterResourceWhitelist: []
```

Audit

Argo CD emits events and API logs for logins, syncs and changes. Ship them to central logging and alert on admin-level actions and manual syncs of production. Git history shows what changed, and Argo logs show who triggered it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Argo CD: SSO, RBAC and audit fits together

Part	What it does	What goes wrong
SSO	Store the client secret in the Argo CD secret, not in the config map.	Shared admin credentials.
RBAC	Policies read as subject, resource, action, object.	Giving all developers sync rights in production.
Projects	An AppProject limits which Git repositories an application may use, which clusters and namespaces it may deploy to and which Kubernetes kinds it may create.	Projects that allow any repository and cluster.
Audit	Argo CD emits events and API logs for logins, syncs and changes.	No central audit logs.

Flow: SSO to RBAC / RBAC to Projects / Projects to Audit

Incident: Who synced production?

Incident: *A bad sync happened and no one could say who triggered it.*

```
application: shop-prod
operation: sync initiated by admin
revision: HEAD   health: Degraded
```

Everyone used the shared admin login. The team enabled SSO, disabled admin, gave developers sync rights only on non-production projects and required changes to production to come through reviewed Git commits.

Common mistakes

- Shared admin credentials
- Projects that allow any repository and cluster
- Giving all developers sync rights in production
- No central audit logs

Check yourself

What should you do after SSO works?

Answer: Disable the local admin

What limits where an app can deploy?

Answer: An AppProject

What does a RBAC group mapping do?

Answer: Assigns a role to an identity provider group

Where does Git history help in audit?

Answer: It shows what changed

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Argo CD: notifications and image updates

12 minutes. Send alerts when syncs fail or apps degrade, and automate image tag updates safely through Git instead of editing clusters.

A deployment tool that fails silently is dangerous. And a team that edits version numbers by hand for every release wastes time and makes mistakes. Two small components address these.

What you will learn

- Triggers, templates and subscriptions in notifications
- Routing alerts by environment and severity
- Automating image tag updates through Git
- Keeping automation reviewable

Notifications

```
# argocd-notifications-cm (excerpt)
data:
  service.slack: |
    token: $slack-token
  trigger.on-sync-failed: |
    - when: app.status.operationState.phase in ['Error', 'Failed']
      send: [app-sync-failed]
  template.app-sync-failed: |
    message: |
      Sync of {{.app.metadata.name}} failed: {{.app.status.operationState.message}}

# on an Application
metadata:
  annotations:
    notifications.argoproj.io/subscribe.on-sync-failed.slack: shop-deploys
    notifications.argoproj.io/subscribe.on-health-degraded.slack: shop-deploys
```

A **trigger** decides when, a **template** formats the message and a **subscription** says who receives it. Send routine events to a channel, and page people only for production health problems, to avoid alert fatigue.

Image updates

The **Argo CD Image Updater** or a pipeline step watches a registry and records a new tag in Git, either by committing to the manifest repository or opening a pull request.

```
metadata:
  annotations:
    argocd-image-updater.argoproj.io/image-list: web=registry.example.com/shop/web
    argocd-image-updater.argoproj.io/web.update-strategy: semver
    argocd-image-updater.argoproj.io/write-back-method: git
```

Use a version strategy that fits your release process. For production, prefer pull requests with approval, and use digests when you need an exact immutable reference.

Keep Git as truth

Automatic updates should land as commits so history, review and rollback still work. Never let a tool edit live objects while Git says something else.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Argo CD: notifications and image updates fits together

Part	What it does	What goes wrong
Notifications	A trigger decides when, a template formats the message and a subscription says who receives it.	No notifications for failed syncs.
Image updates	The Argo CD Image Updater or a pipeline step watches a registry and records a new tag in Git, either by committing to the manifest repository or opening a pull request.	Image updaters that bypass Git.
Keep Git as truth	Automatic updates should land as commits so history, review and rollback still work.	Never let a tool edit live objects while Git says something else.

Flow: Notifications to Image updates / Image updates to Keep Git as truth

Incident: The silent Friday failure

Incident: *A failed sync was noticed three days later.*

```
phase: Failed
message: ComparisonError: failed to generate manifest
notifications subscribed: none
```

Production deployments failed over the weekend and customers kept seeing the old version. The team added failure notifications to a team channel, paging for production degraded health, and a dashboard of apps out of sync for over an hour.

Common mistakes

- No notifications for failed syncs
- Paging for every event
- Image updaters that bypass Git
- Using moving tags like latest

Check yourself

What decides when a notification is sent?

Answer: A trigger

How should image updates reach clusters?

Answer: Through commits in Git

Why avoid latest tags?

Answer: They are not reproducible

How avoid alert fatigue?

Answer: Route by environment and severity

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Progressive delivery with Argo Rollouts

13 minutes. Release gradually with Argo Rollouts canary and blue-green strategies, automated analysis and quick aborts when metrics go bad.

A rolling update checks readiness, not user experience. Progressive delivery adds real traffic shifting and metric analysis, so you can release often with less fear.

What you will learn

- Canary and blue-green strategies
- Defining steps, weights and pauses
- Automated analysis against metrics
- Aborting and promoting

A canary rollout

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: web
spec:
  replicas: 6
  selector:
    matchLabels: {app: web}
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: {duration: 5m}
        - analysis:
            templates:
              - templateName: error-rate
        - setWeight: 50
        - pause: {duration: 10m}
        - setWeight: 100
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
        - name: web
          image: registry.example.com/shop/web:1.9.0
```

The canary receives a small share, then more if checks pass. A traffic router such as an ingress controller or mesh gives accurate percentages, otherwise weights follow pod counts.

Analysis

```
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: error-rate
spec:
  metrics:
    - name: error-rate
      interval: 1m
      count: 5
      failureLimit: 1
      successCondition: result[0] < 0.02
      provider:
        prometheus:
          address: http://prometheus.monitoring:9090
          query: sum(rate(http_requests_total{app="web",code=~"5.."}[2m])) / sum(rate(http_request \
    s_total{app="web"}[2m]))
```

MARGA by EknathaLabs
marga.eknathalabs.com

If the metric breaches the condition, the rollout **aborts** and traffic returns to the stable version.

Operate

```
kubectl argo rollouts get rollout web --watch
kubectl argo rollouts promote web
kubectl argo rollouts abort web
```

Blue-green keeps the full old version running and switches all traffic at once after checks, which uses double resources but allows instant rollback. Use Argo CD to sync the Rollout from Git, and keep database changes backward compatible, because traffic may run both versions.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Progressive delivery with Argo Rollouts fits together

Part	What it does	What goes wrong
A canary rollout	The canary receives a small share, then more if checks pass.	Weights that follow pod counts without a traffic router.
Analysis	If the metric breaches the condition, the rollout aborts and traffic returns to the stable version.	Canary with no automated analysis.
Operate	Blue-green keeps the full old version running and switches all traffic at once after checks, which uses double resources but allows instant rollback.	Database changes that break the old version.

Flow: A canary rollout to Analysis / Analysis to Operate

Incident: The canary that stopped itself

Incident: *A release with a bug was aborted automatically at 10 percent.*

```
step 3/6 analysis: error-rate = 0.061
successCondition result[0] < 0.02 failed
Rollout aborted: canary scaled to 0, stable at 100%
```

A change introduced a bug that affected one payment path. The analysis caught the error rate within five minutes, and only a tenth of traffic was affected. The team fixed the bug, added a test and kept the same analysis for every release.

Common mistakes

- Canary with no automated analysis
- Weights that follow pod counts without a traffic router
- Database changes that break the old version
- Pause durations too short to see problems

Check yourself

What does Argo Rollouts replace?

Answer: The Deployment object

What does analysis do?

Answer: Checks metrics to decide to continue or abort

What is blue-green?

Answer: Switch traffic between two full versions

Why keep schema changes compatible?

Answer: Both versions may run at once

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. Argo CD: disaster recovery and upgrades

12 minutes. Back up and rebuild Argo CD, recover from a lost cluster using Git, and upgrade Argo CD with care for CRDs and breaking changes.

A Git-driven delivery tool has a pleasant recovery story, because applications are described in a repository. It is not automatic, though: you still need to protect Argo CD's own settings, secrets and cluster credentials, and upgrade it carefully.

What you will learn

- What is in Git and what is not
- Backing up Argo CD configuration and secrets
- Rebuilding from a bootstrap application
- Upgrading safely

What to protect

Application manifests live in Git. Not in Git: cluster credentials, repository credentials, SSO client secrets, TLS certificates and any application or project objects created by hand. Put the rest of the configuration in Git too, and keep the secrets in a secret manager.

Backup

```
argocd admin export -n argocd > argocd-backup-$(date +%F).yaml
kubectl get secret -n argocd -l argocd.argoproj.io/secret-type=cluster -o yaml > clusters.yaml
```

Encrypt backup files and store them off-cluster. Test that the export can be imported into a fresh instance.

Rebuild

```
kubectl create namespace argocd
kubectl apply -n argocd -f install.yaml
kubectl apply -f bootstrap/root-app.yaml     # app of apps that creates everything else
argocd app list
```

With the **app-of-apps** pattern, one root application points to a directory of other applications, so applying that single object restores the whole fleet. Application data in volumes is not in Git and needs its own backups.

Upgrade

Read release notes for breaking changes and CRD updates, since Helm does not always upgrade CRDs. Upgrade one minor version at a time, first in a staging instance with a copy of production configuration. After the upgrade, check applications for new diffs or health changes, and keep the previous manifests to roll back.

```
argocd version
kubectl get applications -n argocd -o wide | grep -v Synced
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Argo CD: disaster recovery and upgrades fits together

Part	What it does	What goes wrong
What to protect	Application manifests live in Git. Not in Git: cluster credentials, repository credentials, SSO client secrets, TLS certificates and any application or project objects...	Credentials that exist only in the live cluster.
Backup	Encrypt backup files and store them off-cluster.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Rebuild	With the app-of-apps pattern, one root application points to a directory of other applications, so applying that single object restores the whole fleet.	Never rehearsing a rebuild.
Upgrade	Read release notes for breaking changes and CRD updates, since Helm does not always upgrade CRDs.	Skipping minor versions in upgrades.

Flow: What to protect to Backup / Backup to Rebuild / Rebuild to Upgrade

Incident: A rebuilt cluster with no repository access

Incident: *Argo CD came back but could not read private repositories.*

```
ComparisonError: authentication required
rpc error: repository not found
31 applications in Unknown state
```

The Git credentials had been added through the UI and were not stored anywhere else. After rebuilding, Argo CD had no access to private repositories. The team moved credentials into a secret manager and declarative secrets, and rehearsed a full rebuild twice a year.

Common mistakes

- Credentials that exist only in the live cluster
- Not backing up Argo CD secrets
- Skipping minor versions in upgrades
- Never rehearsing a rebuild

Check yourself

What does app-of-apps help with?

Answer: Recreating all applications from one root

What is not in Git by default?

Answer: Cluster and repository credentials

Why upgrade one minor version at a time?

Answer: To avoid breaking changes

What also needs backup?

Answer: Application data volumes

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Continuous delivery with Argo CD

Applications, sync policies, hooks, projects, ApplicationSets and a debugging checklist.

An Application

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api
  namespace: argocd
spec:
  project: team-shop
  source:
    repoURL: https://example.com/platform/shop-config.git
    targetRevision: main
    path: apps/shop-api
  destination:
    server: https://kubernetes.default.svc
    namespace: shop
```

Warning: *The default project is unrestricted. Give each team its own project with limited sources and destinations.*

Everyday commands

```
argocd app list
argocd app get shop-api
argocd app diff shop-api
argocd app sync shop-api --dry-run
argocd app history shop-api
```

Warning: *Hand edits to live resources are reverted when self-heal is on. Change the repository instead.*

Automated sync

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
  syncOptions:
    - CreateNamespace=true
```

Warning: *Prune deletes whatever disappears from the source. A wrong path can look like delete everything. Protect stateful resources with Prune=false.*

Waves and hooks

```
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "-1"
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: HookSucceeded
```

Warning: *Hooks can run more than once on retries. Make migrations idempotent.*

ApplicationSet

```
generators:
  - list:
      elements:
        - env: staging
        - env: production
template:
  metadata:
    name: "shop-{{env}}"
```

MARGA by EknathaLabs
marga.eknathalabs.com

Ignore expected differences

```
ignoreDifferences:
  - group: apps
    kind: Deployment
    jsonPointers:
      - /spec/replicas
```

Debug stages

```
1. Fetch   - repo access, revision, path
2. Render  - helm template / kustomize build with the same values
3. Apply   - permissions, project rules, webhooks, CRD order
4. Health  - ordinary Kubernetes debugging on the resources
```

Warning: *Never paste a secret into a values file. Reference an external secret store and rotate anything already committed.*

MARGA by EknathaLabs
marga.eknathalabs.com